Deep-Reinforcement-Learning-Based Task Offloading and Resource Allocation in Mobile Edge Computing Network With Heterogeneous Tasks

Tao Jiang, *Fellow, IEEE*, Zhaoping Chen, Zilong Zhao, Mingjie Feng, *Member, IEEE*, and Jiaxi Zhou

***Abstract*—The proliferation of intelligent Internet of Things (IoT) applications has led to an increase in the complexity of tasks generated by IoT devices putting pressure on the timely execution of these tasks. Mobile edge computing (MEC) has emerged as a promising paradigm to deliver low-latency computing services, enabled by task offloading from users to MEC servers. Meanwhile, as the IoT applications become increasingly diversified, the demand for communication and computing resources significantly varies over different tasks, highlighting the importance of efficient task offloading and resource allocation strategies in supporting low-latency task processing. Considering the heterogeneity of tasks, this article investigates the problem of task offloading and resource allocation strategies in the MEC system with heterogeneous tasks and propose a deep reinforcement learning (DRL)-based solution. Specifically, we consider task offloading strategies across various combinations of different task types and focus on optimizing channel allocation to minimize task completion delay. The effectiveness of proposed approach in reducing task completion latency is demonstrated through simulation results.**

***Index Terms*—Deep reinforcement learning (DRL), mobile edge computing (MEC), resource allocation, task offloading.**

I. Introduction

THE EMERGENCE of computational-intensive applications (e.g., virtual reality, natural language processing, and online gaming) has resulted in an surging demand for computational power [1]. Mobile cloud computing (MCC) is an effective approach to satisfy this need by enabling task offloading from various devices to the cloud centers with strong computing power. However, due to multihop transmission between users and cloud centers, task offloading may experience high latency, making MCC unsuitable for latency-sensitive Internet of Things (IoT) applications. In response to this drawback, mobile edge computing (MEC) has been proposed as a novel computing paradigm [2], [3], [4]. MEC brings computing resources closer to users by deploying servers at the network edge, thereby reducing the transmission delays for task offloading. To implement MEC, future communication networks are expected to support a dense deployment of edge nodes (ENs), implemented by equipping small base stations and wireless access points, with servers to deliver computing services. This paradigm enables the offloading of delay-sensitive tasks to these ENs, thereby facilitating low-latency, high-performance computing for mobile devices [5].

With the advancement of mobile communications and IoT, the number of devices supported by MEC systems is expected to increase significantly [6]. This surge will intensify competition for the limited computing and communication resources among devices, leading to inadequate resource available to individual devices, thereby challenging the MEC system's ability to deliver low-latency computing services. To address this issue, considerable research has been devoted, with focuses on optimizing the task offloading strategies and resource allocation among users [7], [8], [9], [10].

Note that many of these studies focus on scenarios with homogeneous tasks, where the tasks to be processed belong to the same type. Meanwhile, the diversification of IoT devices and applications has led to more complex tasks varied computational requirements [6]. As a result, traditional homogeneous task oriented task offloading and resource allocation strategies cannot be applied to achieve satisfactory performance, necessitating adaptive strategies that can accommodate different types of tasks.

The implementation of task-aware strategies faces several challenges. First, the dynamic nature of user demands results in tasks of various types generated at unpredictable times, which increases the difficulty of devising an intelligent strategy that can adapt to the dynamic demands.

Second, as higher frequency will be used in future wireless systems and many future devices are characterized as high mobility (e.g., drones and vehicles), the wireless channel used for task offloading will be more dynamic. Thus, traditional static resource allocation strategies cannot capture such dynamics and fail to achieve to satisfactory performance. Finally, with limited resources, it is crucial to maximize the utilization of system resources to enhance system performance, which necessitates a joint optimization of both task offloading and resource allocation. However, such an optimization problem is inherently complex and is generally classified as

Received 25 September 2024; revised 13 November 2024; accepted 5 December 2024. Date of publication 9 December 2024; date of current version 9 April 2025. This work was supported by the National Natural Science Foundation of China under Grant 62101202. *(Corresponding authors: Tao Jiang; Mingjie Feng.)*

Tao Jiang, Zhaoping Chen, Zilong Zhao, and Jiaxi Zhou are with the Research Center of 6G Mobile Communications and the School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: tao.jiang@ieee.org; d202287045@hust.edu.cn; zilongzhao@hust.edu.cn; zhoujiaxi@hust.edu.cn).

Mingjie Feng is with the Research Center of 6G Mobile Communications and the Wuhan National Laboratory for Optoelectronics, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: mingjiefeng@hust.edu.cn).

Digital Object Identifier 10.1109/JIOT.2024.3514108

2327-4662 © 2024 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: UNIVERSITY OF ULSTER. Downloaded on April 13,2026 at 19:02:17 UTC from IEEE Xplore. Restrictions apply.

NP-hard. Consequently, solving it in polynomial time is infeasible, making traditional optimization techniques inefficient or even infeasible.

In the presence of these challenges, we employ deep reinforcement learning (DRL) to design efficient task offloading and resource allocation strategies. DRL has proven to be a highly effective approach for complex decision-making problems in dynamic environments [17], [18], [19], [20]. Reinforcement learning (RL) algorithms utilize historical experiences to enable intelligent decision making. However, traditional RL algorithms, such as Q-learning, struggle with high-dimensional problems due to the requirement of storing Q-values (i.e., the expected reward for taking various actions under different system states). In contrast, DRL overcomes this limitation by integrating RL with the feature extraction capabilities of deep learning (DL). This combination facilitates effective decision making in the presence of high-dimensional environmental features. The fundamental approach of implementing DRL is deep Q-networks (DQNs), where a neural network is utilized to approximate the Q-values, thereby effectively managing the complexities associated with high-dimensional feature spaces.

In this article, we investigate the problem of joint optimization of task offloading and channel selection for MEC systems with heterogeneous tasks, with the objective of minimizing the average task completion delay. In particular, we employ the proactive and adaptive decision-making capabilities of DRL to accommodate the channel dynamics and task heterogeneity, and propose DRL-based solutions accordingly. The contributions of this article are as follows.

1) Considering the task heterogeneity, we formulate the joint optimization problem of task offloading and resource allocation in MEC systems with heterogeneous tasks. Specifically, we optimize the task partitioning ratio and the channel average task completion delay.
2) We transform the formulated problem into a Markov decision process (MDP) and propose a DRL-based solution to generate intelligent strategies that are adaptive to channel dynamics and task heterogeneity. Specifically, we train a DQN with an experience replay mechanism to facilitate proactive and efficient decision making, which can be applied to large-scale problems.
3) The effectiveness of the proposed method was validated through simulation experiments. The results demonstrate that our approach achieves lower latency compared to the three baseline methods. Additionally, our strategy proves to be optimal across various task types and densities.

II. Related Work

A. Task Offloading and Resource Allocation in Mobile Edge Computing

Task offloading and resource allocation strategies have been proposed to reduce task completion delay and device energy consumption. In [11], task offloading in a multiuser, multiserver MEC environment was investigated, focusing on optimizing both the average response time and the total energy consumption, thereby significantly enhancing user experience and system utility. Based on federated learning framework, a distributed optimization scheme was proposed in [12], aimed at minimizing the overall cost of the MEC system by dynamically adjusting task offloading ratios, bandwidth allocation, and computing power distribution. Similarly, Tong et al. [13] formulated task offloading as a stochastic optimization problem using a virtual queue model, with an emphasis on balancing energy consumption and queue backlog under varying task arrival rates. A fairness-aware task offloading and resource allocation scheme was proposed in [14], aiming to achieve a good tradeoff between performance and fairness in the presence of constrained resources. Tan et al. [15] designed intelligent offloading strategies under the context of 5G and IoT, which were shown to enhance overall system performance with reduced response time to the delay-sensitive applications. In [16], a joint optimization scheme that integrates content caching was proposed to improve the efficiency of computational offloading and reduce task completion latency. However, these studies do not account for the heterogeneity of tasks and the dynamics of the varying environment, making them unfit for MEC systems operating in highly dynamic environments with various types of tasks. In contrast, this article apply DRL to accommodate the time-varying task demands and environments, resulting in improved resource utilization and system performance.

B. Application of Deep Reinforcement Learning in Mobile Edge Computing

Due to the capability of intelligent decision making, DRL has been applied in MEC systems in recent years. Asheralieva and Niyato [18] presented a framework that integrates the game theory and Bayesian DRL to enhance task processing efficiency and overall system profitability. A novel RL-based approach for active sensor selection was proposed in [19], aiming to tackle the challenges of target localization in IoT application scenarios. Additionally, a state-space reduction technique was introduced to improve the efficiency of computational node placement and selection, thereby optimizing the system performance.

In [20], a DRL-based algorithm was designed for collaborative mobile computing offloading in a three-layer MEC network. The proposed solution utilizes the prioritization of experience samples in the experience pool, to enhances the efficiency of task offloading strategies in complex network environments. A multiagent DRL-based offloading scheme was proposed in [21] to accommodate the inherent uncertainties of the environment. The goal is to optimize the offloading decisions of multiple agents to minimize total task processing latency. The problem of task offloading in an MEC system with multiple connections was investigated in [22], where a multiagent DRL-based algorithm was proposed to enable efficient collaboration among the edge devices, aiming to minimize the computational latency. Although DRL was successfully applied in the above works, the schemes were designed for homogeneous tasks. In contrast, our scheme can accommodate different types of tasks and fully consider the dynamics of MEC systems.

Authorized licensed use limited to: UNIVERSITY OF ULSTER. Downloaded on April 13,2026 at 19:02:17 UTC from IEEE Xplore. Restrictions apply.

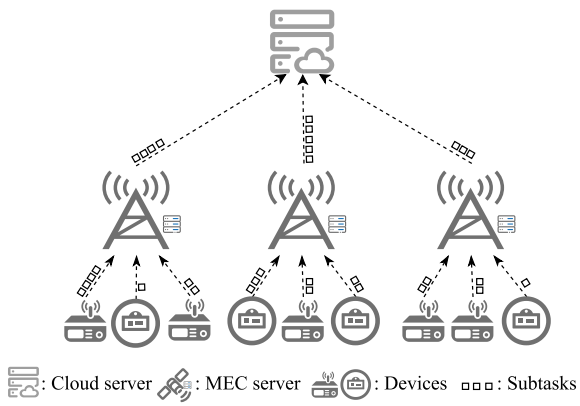

Fig. 1. System model of the MEC system with one cloud server and multiple edge severs.

III. System Model

A. Network Architecture

As illustrated in Fig. 1, we consider an MEC system comprising N mobile devices, a remote cloud server, and M MEC servers integrated into base stations. Each base station, along with its associated edge server, is referred to as an EN. We denote the set of all ENs as $m \in \mathcal{M} = \{1, 2, \ldots, M\}$. These ENs collectively serve N devices, which are represented by the set $n \in \mathcal{U} = \{1, 2, \ldots, N\}$. Each device connects to an EN via a wireless link, while the ENs are connected to the remote cloud server (denoted by s_0) through high speed wired links. The connection between devices and ENs is indicated by the matrix $\mathcal{X} = \{x_{n,m} \mid n = 1, 2, \ldots, N; m = 1, 2, \ldots, M\}$, where each element is defined by the following binary variable:

$$x_{n,m} = \begin{cases} 1, \text{ if device } n \text{ is connected to EN } m \\ 0, \qquad\qquad \text{otherwise} \end{cases} \quad n \in \mathcal{U}, m \in \mathcal{S}. \tag{1}$$

Each device generates a single task during each time slot. Thus, the tasks are also indexed by $n = 1, \ldots, N$. The tasks generated by each device u_n are independent of one another. Without loss of generality, we consider four types of tasks to reflect the task heterogeneity. The tasks are classified according to their input data type, including text, image, audio, and video. Each task n is characterized by $M_n(t) = \kappa_n(t), d_n(t), p_n(t)$, where $\kappa_n(t)$ denotes the task type (0 for text, 1 for image, 2 for audio, and 3 for video), $d_n(t)$ is the size of the input data of task n, and $p_n(t)$ is the number of CPU cycles required to complete the task. We consider the scenario where each device can connect to at most one EN. Thus, $\sum_{m=1}^{M} x_{n,m} \leq 1$ must be satisfied for $n \in \mathcal{U}$. For devices connected to EN m, their tasks can be offloaded to EN m and/or forwarded by EN m to the cloud server for processing. It is assumed that each task can be partitioned into multiple portions of subtasks and executed separately. We denote the partitioning strategy for device u_n as $(\alpha_n, \beta_n, \gamma_n)$, where α_n, β_n, and γ_n represent the ratios of the task assigned to device u_n, the associated EN, and the cloud server, respectively. Then, the offloading ratio constraints for device u_n is given by

$$\alpha_n + \beta_n + \gamma_n = 1, n \in \mathcal{U}. \tag{2}$$

B. Communication Model

Orthogonal time-frequency resource allocation, such as orthogonal frequency division multiple access (OFDMA), is utilized to support task offloading from multiple devices to the same EN, as employed in LTE and 5G systems. The set of available channels is denoted by $\mathcal{H}$, indexed by $1, 2, \ldots, N_c$. And the device n communicate with EN via channel h_n. We assume that communication resources (i.e., channels) are evenly distributed among all devices connected to an EN. Additionally, each EN can measure the uplink signal-to-interference-plus-noise ratio (SINR) of devices connected to it. Let B represent the bandwidth of each channel, and let $S_{n,m}$ denote the uplink SINR from device n to EN m. The data rate of device u_n is calculated as

$$R_{n,m} = b \log_2\big(1 + S_{n,m}\big). \tag{3}$$

Spectrum reuse across different ENs is allowed to enhance spectral efficiency. Consequently, the interference between different ENs is considered in this model.

Let b_n be the size of the input data of task n measured in bits. Given the partitioning ratio, the offloading time is calculated as follows.

1) Device-EN Offloading Time: The uplink SINR from device n to the EN m is as follows:

$$\begin{aligned} \mathrm{SINR}_{n,m} &= \frac{p_n L\big(d_{n,m}\big)}{I_{n,j} + \delta^2} \\ I_{n,j} &= \sum_{j=1, j\neq m}^{M} p_j L\big(d_{n,j}\big) \end{aligned} \tag{4}$$

where p_n indicates the transmission power of device n; δ^2 is the additive white Gaussian noise (AWGN) power; $I_{n,j}$ is the interference introduced by all other devices other than device n; p_j is the transmission power of BS j; $L(\cdot)$ is the path loss function; and $d_{n,m}$ is the distance between EN m to device n in meters [23].

The path loss between an BS and the device is given by

$$L(d) = 37 + 30 \log_{10}(d) \text{ dB}. \tag{5}$$

Combining (3) and (4), we can calculate the transmission rate between device u_n and EN s_m as (6)

$$R_{n,m}^{(E)} = b \cdot \log_2\left(1 + \frac{p_m L(d_{n,m})}{I_{n,j} + \delta^2}\right). \tag{6}$$

Given the task-partitioning strategy $(\alpha_n, \beta_n, \gamma_n)$ of device n, the uplink transmission time of device n when offloading to EN m is expressed as

$$t_{\mathrm{comm},n}^{(E)} = \frac{(\beta_n + \gamma_n) d_n}{R_n^{(E)}} = \frac{(\beta_n + \gamma_n) d_n}{b \cdot \log_2\left(1 + \frac{p_m L(d_{n,m})}{I_{n,j} + \delta^2}\right)}. \tag{7}$$

2) EN-Cloud Offloading Time: Given the transmission rate of wired link c, the transmission time for this process can thus be formulated as follows:

$$t_{\mathrm{comm},n}^{(C)} = \frac{\gamma_n d_m}{c}. \tag{8}$$

Authorized licensed use limited to: UNIVERSITY OF ULSTER. Downloaded on April 13,2026 at 19:02:17 UTC from IEEE Xplore. Restrictions apply.

C. Computation Model

1) Local Device Computing Time: Denote the computational ability of device u_n as $c_n^{(L)}$, which is measured in CPU cycles per second. With the partitioning ratio α_n, the number of CPU cycles required to complete the subtasks assigned to the local device is $\alpha_n d_n p_n$. Then, the execution time (in seconds) at device n is expressed

$$t_{\text{comp},n}^{(L)} = \frac{\alpha_n d_n p_n}{c_n^{(L)}}. \tag{9}$$

2) MEC Server Computing Time: We assume that each EN is equipped with a multicore processor, enabling it to concurrently process multiple tasks received from various devices. The computational capacity of an EN is distributed equally among all devices connected to that EN within each time slot. Once a device uploads its task data to an EN, the EN processes these tasks immediately.

The traffic load of EN m is defined as the number of devices connected to it, denoted by $L_m = \sum_{n=1}^{N} x_{n,m}$. The computational capability of EN m is represented by $c_m^{(E)}$, measured in CPU cycles per second. Since this computational capacity is evenly distributed among all devices connected to the EN, the computational capacity allocated to device n by EN m is given by $c_{n,m}^{(E)} = (c_m^{(E)}/L_m)$. Given the partitioning ratio β_n, the execution time for the device n at EN m is given by

$$t_{\text{comp},n}^{(E)} = \frac{\beta_n d_n p_n \cdot L_m}{c_m^{(E)}}. \tag{10}$$

3) Cloud Server Computing Time: The cloud server provides a fixed computational capacity to device n, given by $c_n^{(C)}$, which is determined by the service purchased by device n. Recall that the ratio of subtasks of user n processed by the cloud server is γ_n, the execution time for device n at the cloud server is given by

$$t_{\text{comp},n}^{(C)} = \frac{\gamma_n d_n p_n}{c_n^{(C)}}. \tag{11}$$

IV. Problem Formulation

The total time completing task at the local device, connected EN, and cloud server are denoted by $t_n^{(L)}$, $t_n^{(E)}$, and $t_n^{(C)}$, respectively. Then, we have

$$\begin{aligned} t_n^{(L)} &= t_{\text{comp},n}^{(L)} \\ t_n^{(E)} &= t_{\text{comm},n}^{(E)} + t_{\text{comp},n}^{(E)} \\ t_n^{(C)} &= t_{\text{comm},n}^{(E)} + t_{\text{comm},n}^{(C)} + t_{\text{comp},n}^{(C)}. \end{aligned} \tag{12}$$

Because tasks can be concurrently executed in IIoT device, EN, and cloud server. Thus, the latency for completing the whole task is given by

$$T_n = \max\{t_n^{(L)}, t_n^{(E)}, t_n^{(C)}\}. \tag{13}$$

We assume that each device is associated its nearest EN. Thus, the matrix $\mathcal{X}$ is determined by the locations of devices and ENs.

We aim to minimize the average task completion latency of all IIoT devices, which is equivalent to minimize the sum latency of all devices. Let $\boldsymbol{\alpha}$, $\boldsymbol{\beta}$, $\boldsymbol{\gamma}$, and $\boldsymbol{h}$ denote the vector $[\alpha_n]_{n\in\mathcal{U}}$, the vector $[\beta_n]_{n\in\mathcal{U}}$, the vector $[\gamma_n]_{n\in\mathcal{U}}$, and the vector $[h_n]_{n\in\mathcal{U}}$, respectively. $\boldsymbol{\alpha}$, $\boldsymbol{\beta}$, and $\boldsymbol{\gamma}$ represent the proportion of tasks processed by local, edge, and cloud servers, respectively. And this directly affects the completion latency of the entire task. In order to minimize the task completion latency, we need to optimize the parameters $\boldsymbol{\alpha}$, $\boldsymbol{\beta}$, and $\boldsymbol{\gamma}$. So, such a problem is formulated as

$$\mathbf{P}: \min_{\{\boldsymbol{\alpha},\boldsymbol{\beta},\boldsymbol{\gamma},\boldsymbol{h}\}} \sum_{n=1}^{N} T_n \tag{14}$$

$$\text{s.t.} \quad \alpha_n + \beta_n + \gamma_n = 1, \ n \in \mathcal{U} \tag{14a}$$

$$0 \le \alpha_n, \beta_n, \gamma_n \le 1, \ n \in \mathcal{U} \tag{14b}$$

$$h_n \in \{1, 2, \ldots, N_c\}, \ n \in \mathcal{U} \tag{14c}$$

$$h_i \ne h_j, \ i, j \in \mathcal{U}, m \in \mathcal{S}, x_{i,m} = x_{j,m} = 1 \tag{14d}$$

$$\sum_{n=1}^{N} c_{n,m}^{(E)} \le c_{\max,m}^{(E)} \tag{14e}$$

$$p_n \le p_{n,\max}, \ n \in \mathcal{U}. \tag{14f}$$

The constraints in (14a) and (14b) are directly from the definition of $\boldsymbol{\alpha}$, $\boldsymbol{\beta}$, and $\boldsymbol{\gamma}$; constraint (14c) specifies the set of channels available to each user; constraint (14d) comes from the fact that the devices connected to the same EN cannot use the same channel; constraint (14e) indicates that the sum computational resources allocated to various devices by an EN cannot exceed its maximum available CPU cycle frequency $c_{\max}^{(C)}$; and finally, the constraint in (14f) constrains the transmission power of each device.

V. Solution Algorithms

A. MDP Model

The optimization problem $\mathbf{P}$ is transformed into an MDP presented as follows.

State: The task state of all available nodes and the state of channels are defined as the state. The state of a device at any time t is represented by an array composed of the real-time state of task, and the channel state. For the device u_n, the real-time state of task is given by $s_n^1(t) = [\kappa_n(t), d_n(t), p_n(t)]$.

Suppose device n connects with EN m. As EN can measure the channels of all its associated devices, the devices connected to the same EN can obtain the channel state of each other. Let $s_n^2(t) = [h_{n,1}, \ldots, h_{n,i}, \ldots, h_{n,N_c}]$ be the channel related state of device u_n, where $h_{n,i} = 0$ represents the channel i is not occupied, otherwise is occupied by other device connected with s_m. Then, the state of device u_n is defined as

$$\begin{aligned} s_n(t) &= \left[s_n^1(t); s_n^2(t)\right] \\ &= \left[\kappa_n(t), d_n(t), p_n(t), h_{n,1}, \ldots, h_{n,N_c}\right]. \end{aligned} \tag{15}$$

So, the global state is given by

$$s(t) = [s_1(t), s_2(t), \ldots, s_N(t)]. \tag{16}$$

Action: The task offloading and channel selection actions of device N can be represented as an array, given by

$$\begin{aligned} a_n(t) &= [\alpha_n, \beta_n, h_n], \\ a(t) &= [a_1(t), a_2(t), \ldots, a_N(t)]. \end{aligned} \tag{17}$$

Authorized licensed use limited to: UNIVERSITY OF ULSTER. Downloaded on April 13,2026 at 19:02:17 UTC from IEEE Xplore. Restrictions apply.

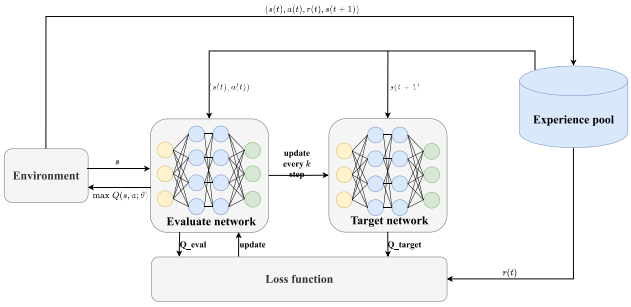

Fig. 2. Process of DQN training.

Reward: Since we aim to minimize the total latency, the reward $r(t)$ can be given by

$$r(t) = -\sum_{n=1}^{N} T_n. \tag{18}$$

B. DQN Algorithm

Traditional algorithm studies face challenges, such as the interplay between computation and communication resources and difficulties with high-dimensional state data, leading to inefficiencies. RL, a subset of machine learning, aims to maximize cumulative rewards by training agents to take optimal actions in various environments. RL involves an agent, environment, states, actions, and rewards. The agent's actions alter the environment's state, which in turn provides a reward to guide the agent toward an optimal policy through trial-and-error and reward maximization. However, RL struggles with perceiving and processing environmental states efficiently. DL, another machine learning approach, optimizes models by learning deep, nonlinear structures, and dataset features. Integrating DL with RL (DRL) enhances the system's ability to solve problems faster and with improved performance, especially in large-scale network environments, by overcoming RL's environmental perception weaknesses.

DQN is a DRL algorithm that aims to determine the best course of action in a given state. The optimal action-value function, $Q^*(s, a)$, represents the highest expected return when taking action a in state s following a policy π. The Bellman equation updates this function as $Q^*(s, a) = r + \lambda \max Q^*(s', a')$, where λ is the discount factor that adjusts the importance of future rewards. In DQN, a neural network approximates the action-value function, denoted as $Q(s, a; \theta) \approx Q^*(s, a)$. Consequently, the formula for the Q-target in DQN can be formulated as

$$Q^{\text{target}}(s, a; \theta) = r + \lambda \max Q^{\text{target}}\left(s', a'; \theta'\right). \tag{19}$$

C. Training Process of DQN Model

Each device acts as an agent who interacts with the environment and make decisions based on its DQN. The details of training the DQN of each device, are shown in Fig. 2. The parameters of evaluate network and target network are given by θ and θ', respectively.

1) Initialization: We initialize the experience pool and the parameter θ and θ'.

2) Action and Feedback: In this step, an ε-greedy strategy is used to select an action, denoted as

$$a = \begin{cases} \arg\max_a \ Q(s, a; \theta), & \epsilon \le e \le 1 \\ \text{randomly selection}, & 0 \le e \le \epsilon \end{cases} \tag{20}$$

where $e(0 \le e \le 1)$ is a random value.

We use a tuple (s, a, r, s') to represent a experience, where s denotes the state, a denotes the action taken, r is the reward received, and s' is the resulting next state. Experiences collected from interactions with the environment are stored in an experience pool.

3) Training Evaluate Network: During training, experiences are randomly sampled from this pool to update the evaluate network. This random sampling helps reduce the correlation between consecutive experiences, thus improving the stability and efficiency of the learning process.

In iteration i, the Q-network is trained by minimizing a loss function. The loss function used in DQN is the mean-squared error (MSE) between the predicted Q-values and the target Q-values. This can be expressed as

$$L(\theta) = \mathbb{E}\left[\left(r + \lambda \max_{a'} Q(s', a'; \theta_{i-1}) - Q(s, a; \theta_i)\right)^2\right] \tag{21}$$

where the parameter θ_{i-1} of the previous iteration is fixed.

4) Updating Target Network: After every k iterations, the parameter of the Q-evaluation network are all copied to the Q-target network.

D. Complexity and Convergence Analysis

We commence with a complexity analysis of the DQN algorithm. Let τ represent the total number of training iterations. The computational efficiency of the proposed algorithm for each training iteration is derived based on the complexity of neural network parameters. In the computation of neural network parameters, agents utilize fully connected neural networks to generate actions. Denote K as the number of layers in the fully connected neural networks. Let ς_k represent the number of neurons in the kth fully connected layer. The time complexity of one training iteration is denoted as $\mathcal{C} = \mathcal{O}(\sum_{k=1}^{K} \varsigma_{k-1}\varsigma_k)$. Consequently, the overall time complexity of the DQN algorithm is $\mathcal{O}(\tau\mathcal{C})$.

The selection of hyperparameters, such as the learning rate and discount rate, plays a crucial role in determining the performance of DRL algorithms. However, confirming the convergence of the proposed DQN method through analytical methods poses a considerable challenge. Presently, the predominant approach in the literature involves determining the optimal hyperparameter configuration for specific problems through empirical trial and error. Consequently, in this article, simulation results are presented in Section VI to facilitate a comprehensive convergence analysis.

VI. Performance Evaluation

In this section, we commence by providing an introduction to the experimental platform and scenario. Subsequently, we proceed to a comprehensive comparison and analysis of the performance of the proposed algorithm under various experimental settings.

Authorized licensed use limited to: UNIVERSITY OF ULSTER. Downloaded on April 13,2026 at 19:02:17 UTC from IEEE Xplore. Restrictions apply.

TABLE I
SIMULATION PARAMETERS

Variable	Parameter	Value
M	Number of EN	5
N	Number of devices	50
P	Transmission power of device	23 dBm
n_0	Noise power density	-70 dBm
B	Bandwidth of each channel	1 MHz
N_c	Number of channels	10
κ	Type of task	{0,1,2,3}
d_n	Data size of task	[256,4096] kB
p_n	Computing complexity of task	[100,500] cycles/bit
$c_n^{(L)}$	Local computation capability	1 GHz
$c_m^{(E)}$	Computation capability of satellite	20 GHz
$c^{(C)}$	Computation capability of cloud	100 GHz

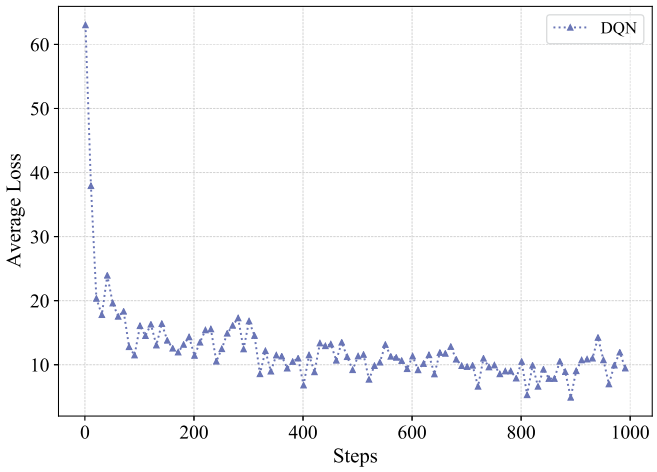

Fig. 3. Convergence of loss function during DQN training.

A. Experiment Settings

The implementation of the DQN algorithm is carried out using Python and PyTorch. The simulation considers an MEC system, comprising varying number of devices, five ENs, and a cloud server. The devices are uniformly distributed in a 600 m × 600 m area. The simulation parameters based on the ones in [24] are summarized in Table I. Notably, the simulations were executed on an Intel Core i5-12400 processor with 32-GB memory, utilizing PyTorch 1.31.1 and Python 3.7.

B. Simulation Results

The proposed solution is denoted as DQN. To showcase the efficacy of the proposed solution, we consider three benchmark schemes. The first scheme, labeled *Random*, involves generating partitioning ratios randomly without employing any intelligent strategy. The second scheme, GA, utilizes a heuristic algorithm. The third scheme, *Q-learning*, is based on storing a Q-value table at each agent. For *Random*, the agent selects random numbers between 0 and 1 as the ratios for task partitioning. For *Q-learning*, the task partitioning ratio is generated in $\{0, 0.1, 0.2, \ldots, 1\}$, so the state space is finite, making Q-learning-based decision making possible. For GA, the crossover and mutation of genes are the core of the algorithm. The gene length is 128, and each optimized variable occupies 32 sites. After the algorithm converges, select the individual with the highest fitness as the final decision. To ensure a fair and equitable comparison, consistent parameter values are maintained across all schemes.

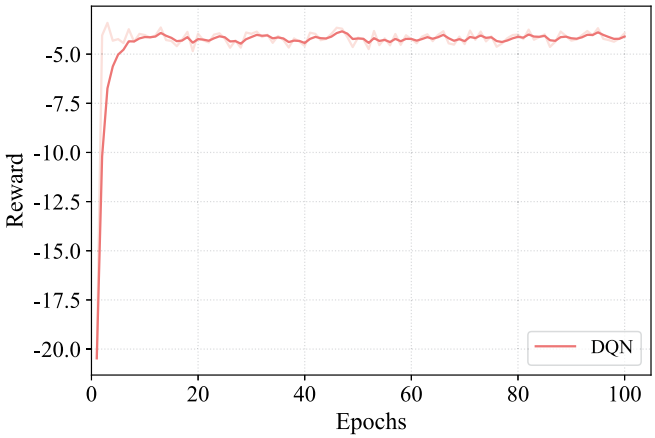

Fig. 4. Reward convergence analysis of DQN.

TABLE II
DIFFERENT SCENARIOS FOR EACH TYPE OF TASK SIZE (KB)

Scenario	Text	Image	Audio	Video
I	256	512	768	1024
II	512	1024	1536	2048
III	768	1536	2304	3072
IV	1024	2048	3072	4096

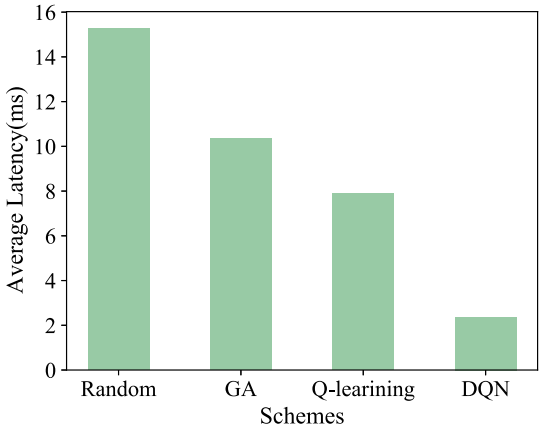

Fig. 5. Average latency under different schemes.

The convergence performance of the DQN algorithm is evaluated and analyzed, where each epoch consists of 100 steps. Figs. 3 and 4 represent the loss convergence and reward convergence of DQN, respectively. As can be seen from the figures, after multiple iterations, the DQN model has basically converged, showing the stability of the DQN model.

Fig. 5 shows the performance of different scheme. The task size of the device in Fig. 5 is given in Table II, and the task density is 30 tasks generated in 100 s. The results demonstrate that *Random* is weaker than the other three algorithms for all types of performance indicators. *Q-learning* and GA showed better performance than *Random* but weaker performance than DQN, because DQN preprocesses a larger number of tasks and uses network model training methods, such as experience replay and target networks. Moreover, the DQN algorithm is able to combine neural networks to analyze the MEC

Authorized licensed use limited to: UNIVERSITY OF ULSTER. Downloaded on April 13,2026 at 19:02:17 UTC from IEEE Xplore. Restrictions apply.

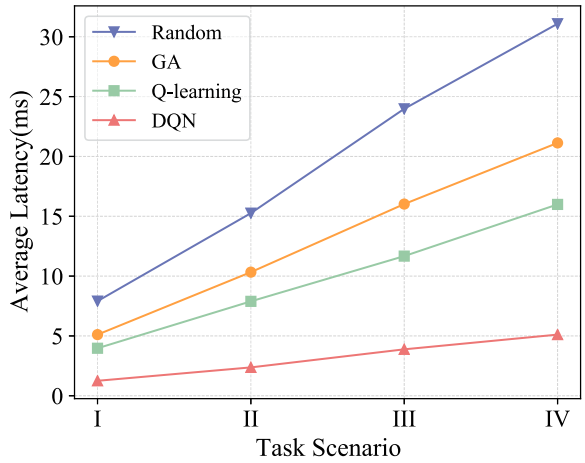

Fig. 6. Average latency versus scenario.

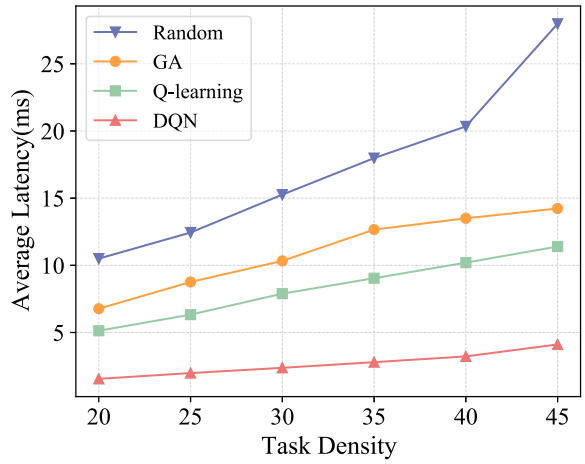

Fig. 7. Average latency versus density.

environment and the task states to obtain a good offloading decision.

Fig. 6 shows the performance of different schemes under varying scenarios. The sizes of the various types of tasks in the experiment are shown in Table II, with a task density of 30 tasks generated in 100 s. The ratios of task types and task sizes generated by each device were consistent. Fig. 6 shows the changing trend of latency for four types of tasks from small to large sizes. Under different task sizes, the latency is optimized by 8%. In Fig. 6, when the task size increases, the average response time increases as the task size increases. This is because the larger the task size, the longer the processing time of the computing nodes and the transmission time in the channel.

In Fig. 7, as the task density increases, four methods are compared. The density of tasks in the experiment is 20, 25, 30, 35, 40, and 45 tasks generated in 100 s. The task size is scenario II in Table II. The ratio of task types and task sizes generated for each device is consistent. An increase in task density is an increase in the number of tasks generated by the device in 100 s, which results in more total tasks and a greater number of model trainings. The task response time is optimized by 15% at different task densities. As the task density increases, the task response time for task transfer and computation increases accordingly. This is because the increase in task density leads to an increase in the number of tasks that the system needs to process at the same time. However, the overall resources of the system are fixed. This leads to a decrease in the resources available for each task and an increase in the task response time.

VII. Conclusion

In this article, we considered the joint optimization of task offloading and resource allocation with the aim of minimizing the task completion latency of devices in MEC systems with heterogeneous tasks. To tackle the challenges of caused by task heterogeneity and system dynamics, we proposed a DRL-based scheme. The proposed scheme was evaluated through simulations, demonstrating its superiority over several benchmark schemes.

References

[1] R. Lohiya and A. Thakkar, "Application domains, evaluation data sets, and research challenges of IoT: A systematic review," *IEEE Internet Things J.*, vol. 8, no. 11, pp. 8774–8798, Jun. 2021.

[2] Q. Li, S. Wang, A. Zhou, X. Ma, F. Yang, and A. X. Liu, "QoS driven task offloading with statistical guarantee in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 21, no. 1, pp. 278–290, Jan. 2022.

[3] Z. Ma et al., "Toward revenue-driven multiuser online task offloading in edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 5, pp. 1185–1198, May 2022.

[4] Y. Miao, G. Wu, M. Li, A. Ghoneim, M. Al-Rakhami, and M. S. Hossain, "Intelligent task prediction and computation offloading based on mobile-edge cloud computing," *Future Gener. Comput. Syst.*, vol. 102, pp. 925–931, Jan. 2020.

[5] F. Li, H. Yao, J. Du, C. Jiang, Z. Han, and Y. Liu, "Auction design for edge computation offloading in SDN-based ultra dense networks," *IEEE Trans. Mobile Comput.*, vol. 21, no. 5, pp. 1580–1595, May 2022.

[6] B. B. Sinha and R. Dhanalakshmi, "Recent advancements and challenges of Internet of Things in smart agriculture: A survey," *Future Gener. Comput. Syst.*, vol. 126, pp. 169–184, Jan. 2022.

[7] Y. Liu, H. Yu, S. Xie, and Y. Zhang, "Deep reinforcement learning for offloading and resource allocation in vehicle edge computing and networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 11, pp. 11158–11168, Nov. 2019.

[8] C. Shang, Y. Sun, H. Luo, and M. Guizani, "Computation offloading and resource allocation in NOMA–MEC: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 10, no. 17, pp. 15464–15476, Sep. 2023.

[9] W. Chu, X. Jia, Z. Yu, J. C. S. Lui, and Y. Lin, "Joint service caching, resource allocation and task offloading for MEC-based networks: A multilayer optimization approach," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2958–2975, Apr. 2024.

[10] Y. Cong, K. Xue, C. Wang, W. Sun, S. Sun, and F. Hu, "Latency-energy joint optimization for task offloading and resource allocation in MEC-assisted vehicular networks," *IEEE Trans. Veh. Technol.*, vol. 72, no. 12, pp. 16369–16381, Dec. 2023.

[11] Z. Tong, X. Deng, F. Ye, S. Basodi, X. Xiao, and Y. Pan, "Adaptive computation offloading and resource allocation strategy in a mobile edge computing environment," *Inf. Sci.*, vol. 537, pp. 116–131, Oct. 2020.

[12] Y. Guo, R. Zhao, S. Lai, L. Fan, X. Lei, and G. K. Karagiannidis, "Distributed machine learning for multiuser mobile edge computing systems," *IEEE J. Sel. Topics Signal Process.*, vol. 16, no. 3, pp. 460–473, Apr. 2022.

[13] Z. Tong, J. Cai, J. Mei, K. Li, and K. Li, "Dynamic energy-saving offloading strategy guided by Lyapunov optimization for IoT devices," *IEEE Internet Things J.*, vol. 9, no. 20, pp. 19903–19915, Oct. 2022.

[14] J. Zhou and X. Zhang, "Fairness-aware task offloading and resource allocation in cooperative mobile-edge computing," *IEEE Internet Things J.*, vol. 9, no. 5, pp. 3812–3824, Mar. 2022.

[15] L. Tan, Z. Kuang, L. Zhao, and A. Liu, "Energy-efficient joint task offloading and resource allocation in OFDMA-based collaborative edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 3, pp. 1960–1972, Mar. 2022.

Authorized licensed use limited to: UNIVERSITY OF ULSTER. Downloaded on April 13,2026 at 19:02:17 UTC from IEEE Xplore. Restrictions apply.

[16] Y. Dong, S. Guo, Q. Wang, S. Yu, and Y. Yang, "Content caching-enhanced computation offloading in mobile edge service networks," *IEEE Trans. Veh. Technol.*, vol. 71, no. 1, pp. 872–886, Jan. 2022.

[17] G. Yang, L. Hou, X. He, D. He, S. Chan, and M. Guizani, "Offloading time optimization via Markov decision process in mobile-edge computing," *IEEE Internet Things J.*, vol. 8, no. 4, pp. 2483–2493, Feb. 2021.

[18] A. Asheralieva and D. Niyato, "Bayesian reinforcement learning and Bayesian deep learning for blockchains with mobile edge computing," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 1, pp. 319–335, Mar. 2021.

[19] M. Shurrab, S. Singh, R. Mizouni, and H. Otrok, "IoT sensor selection for target localization: A reinforcement learning based approach," *Ad Hoc Netw.*, vol. 134, Sep. 2022, Art. no. 102927.

[20] S. Chen, J. Chen, Y. Miao, Q. Wang, and C. Zhao, "Deep reinforcement learning-based cloud-edge collaborative mobile computation offloading in industrial networks," *IEEE Trans. Signal Inf. Process. Netw.*, vol. 8, pp. 364–375, May 2022.

[21] X. Zhu, Y. Luo, A. Liu, M. Z. A. Bhuiyan, and S. Zhang, "Multiagent deep reinforcement learning for vehicular computation offloading in IoT," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9763–9773, Jun. 2021.

[22] Z. Cao, P. Zhou, R. Li, S. Huang, and D. Wu, "Multiagent deep reinforcement learning for joint multichannel access and task offloading of mobile-edge computing in industry 4.0," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6201–6213, Jul. 2020.

[23] X. Huang, S. Tang, Q. Zheng, D. Zhang, and Q. Chen, "Dynamic Femtocell gNB On/Off strategies and seamless dual connectivity in 5G heterogeneous cellular networks," *IEEE Access*, vol. 6, pp. 21359–21368, 2018.

[24] Z. Zhao, M. Feng, C. Ke, Z. Chen, and T. Jiang, "Federated deep recurrent Q-learning for task partitioning and resource allocation in satellite mobile-edge-computing-assisted industrial IoT," *IEEE Internet Things J.*, vol. 11, no. 15, pp. 26444–26458, Aug. 2024.

Authorized licensed use limited to: UNIVERSITY OF ULSTER. Downloaded on April 13,2026 at 19:02:17 UTC from IEEE Xplore. Restrictions apply.